

BACnet/SC Hub (B-SCHUB) - User Manual

Version 1.5.0 · Chipkin Automation Systems · <https://github.com/chipkin/BACnetProfileExample-B-SCHUB-CPP>

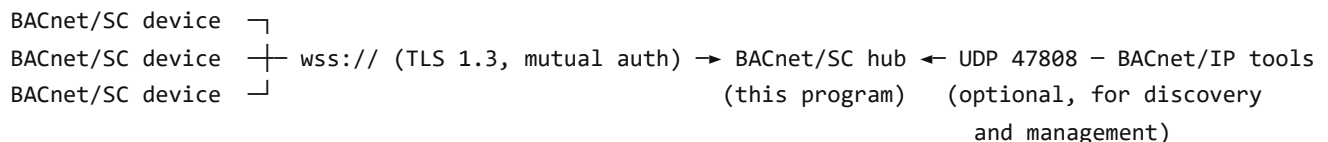
This manual is for people who install, configure and run the hub. `BACnetExampleBSCHUB --version` prints the version you are running, with the CAS BACnet Stack and `common/` helper versions. For how the code works and how to build your own product from it, see [TUTORIAL.md](#).

Contents

1. [What the hub does](#)
2. [Installation](#)
3. [Quick start](#)
4. [Certificates](#)
5. [Configuration](#)
6. [Connecting devices](#)
7. [Managing certificates over BACnet](#)
8. [Status page and HTTP endpoints](#)
9. [Security](#)
10. [Troubleshooting](#)
11. [The BACnet device](#)
12. [Version, licensing and support](#)

1. What the hub does

BACnet Secure Connect (BACnet/SC, ANSI/ASHRAE 135 Annex AB) carries BACnet over TLS-secured WebSockets instead of plain UDP. Devices on a BACnet/SC network don't talk to each other directly: each one connects to a **hub**, which relays their traffic - the role a BACnet/IP broadcast domain plays for UDP devices. This program is that hub, built on the [CAS BACnet Stack](#).



It implements the **B-SCHUB** device profile (ANSI/ASHRAE 135 Annex L):

- **BACnet/SC hub function** (BIBB NM-SCH-B) on Network Port 2: listens on `wss://0.0.0.0:4443/` (TLS 1.3, mutual certificate authentication, WebSocket subprotocol `hub.bsc.bacnet.org`) and relays traffic between the connected devices (up to 4 at a time - see [Connection limit](#)), including this hub's own BACnet device.
- **Optional hub connector**: with `--sc-hub-uri`, the hub also connects out to another BACnet/SC hub (with an optional failover hub).
- **BACnet/IP** on Network Port 1 (UDP 47808), on by default: discovery (Who-Is/I-Am, Who-Has/I-Have), ReadProperty, ReadPropertyMultiple and DeviceCommunicationControl. It can be turned off for a BACnet/SC-only site ([BACnet/SC only](#)).
- **Certificate management over BACnet**: a client can add an issuer and replace the hub's operational certificate (clause 19.8.3).

- **Status page, health and metrics** over HTTP or HTTPS, for people and for monitoring tools.
- **Built-in lab certificate generator**, including a ready-to-import connection file for each device.

This build is for **evaluation and testing**: it accepts at most 4 BACnet/SC devices at a time. For a production hub, contact support@chipkin.com.

Supported BIBBs

BIBB	Name
DS-RP-B	Data Sharing - ReadProperty - B
DS-RPM-B	Data Sharing - ReadPropertyMultiple - B
DM-DDB-B	Device Management - Dynamic Device Binding - B
DM-DOB-B	Device Management - Dynamic Object Binding - B
DM-DCC-B	Device Management - DeviceCommunicationControl - B
NM-SCH-B	Network Management - BACnet/SC Hub Function - B

Services executed

Service	Use
ReadProperty, ReadPropertyMultiple	Read any property of any object.
Who-Is / I-Am, Who-Has / I-Have	Discovery. The hub also broadcasts I-Am at startup over BACnet/IP.
DeviceCommunicationControl	Silence or resume the device, optionally with a password.
AtomicReadFile	Read the certificate File objects.
AtomicWriteFile, WriteProperty	Certificate management only: write a certificate into File 1, 3 or 4, and set their File_Size. No other property is writable.
ReinitializeDevice	ACTIVATE_CHANGES or WARMSTART applies written certificates. Other states are refused.

The [PICS](#) lists every object and property and who answers it.

2. Installation

Download from the [Releases page](#):

Download	What it is
BACnetSCHub-<version>-setup.exe	Windows installer. Installs the program in C:\Program Files\Chipkin\BACnet SC Hub; settings, certificates and logs go in C:\ProgramData\Chipkin\BACnetSCHub. Optionally sets up the BACnetSCHub Windows service (starts on boot, restarts on failure) and firewall rules for UDP 47808 and TCP 4443.

Download	What it is
bacnet-schub-hub_<version>_amd64.deb	Debian/Ubuntu package: <code>sudo apt install ./bacnet-schub-hub_<version>_amd64.deb</code> . The program goes in <code>/opt/bacnet-schub</code> , settings in <code>/etc/bacnet-schub/hub.conf</code> , certificates in <code>/etc/bacnet-schub/certs</code> , logs in <code>/var/log/bacnet-schub</code> , and the bacnet-schub-hub systemd service is enabled and started, running as the <code>bacnethub</code> user.
BACnetSCHub-<version>-linux-x64.tar.gz	The same for other Linux distributions: unpack it and run <code>sudo ./install.sh</code> . <code>sudo ./uninstall.sh</code> removes it (<code>--purge</code> also removes settings and certificates).
BACnetExampleBSCHUB.exe, BACnetExampleBSCHUB	The bare program, to run from a folder of your choice.
manual.pdf, fact-sheet.pdf, PICS.pdf	This manual, the two-page fact sheet, and the PICS.
sbom-windows.cdx.json, sbom-linux.cdx.json	Software bill of materials: the exact CAS BACnet Stack, OpenSSL, libwebsockets and other library versions.
SHA256SUMS.txt	SHA-256 checksums of every download: <code>sha256sum -c SHA256SUMS.txt --ignore-missing</code> .

Both installers make a lab certificate set if the certificate folder is empty, so the hub starts listening straight away; replace it for production (see [Certificates](#)). Settings and certificates are kept when you upgrade or uninstall.

Signed downloads. The Windows program and the Windows installer are code-signed by Chipkin (Azure Artifact Signing, SHA-256 with a timestamp); check them with `Get-AuthenticodeSignature <file>`. Every download - the Linux archive and .deb included - also has a signed build-provenance attestation: `gh attestation verify <file> -repo chipkin/BACnetProfileExample-B-SCHUB-CPP` confirms it was built by this project's release workflow. `SHA256SUMS.txt` lists every file's checksum. See [code-signing.md](#).

Supported platforms

Windows 10/11 and Windows Server 2016 or later (x64). Linux x64: the release build is made on the current Ubuntu LTS, so it needs a distribution with that glibc version or newer; for an older distribution, build from source (see the README). The hub also builds from source on macOS.

The hub runs fine on Windows 10, but BACnet/SC tools on a Windows 10 computer that use Windows' own TLS (such as YABE) can't connect to it or to any other BACnet/SC hub: BACnet/SC requires TLS 1.3, and Windows 10 can't make TLS 1.3 client connections. The hub logs a warning about this when it starts on Windows 10 ([#39](#)).

Firewall

Allow in:

Port	Protocol	For
4443	TCP	BACnet/SC devices connecting to the hub (<code>--sc-port</code>).
47808	UDP	BACnet/IP discovery and management (<code>--port</code>), unless BACnet/IP is off.
8080	TCP	The status page and HTTP endpoints (<code>--http-port</code>) - only if you bind it beyond 127.0.0.1.

The Windows installer can add the first two rules for you.

Running as a service

Windows. The installer sets up the **BACnetSCHub** service. To do it by hand, from an Administrator command prompt:

```
BACnetExampleBSCHUB --install-service --config C:\ProgramData\Chipkin\BACnetSCHub\hub.conf
sc start BACnetSCHub
BACnetExampleBSCHUB --uninstall-service
```

The service runs `BACnetExampleBSCHUB --service --config <file>` with the config file's folder as its working directory, so relative paths in it (`sc-cert-dir = certs`, `log-file = logs\hub.log`) are relative to that folder. There is no console, so use `log-file` to keep a log. The service starts on boot and is restarted after 5, 30 and then 60 seconds if it fails.

Linux (systemd). The `.deb` and `install.sh` install `bacnet-schub-hub.service`, which runs the hub with `/etc/bacnet-schub/hub.conf`, restarts it on failure, and sends its output to the journal as well as `log-file`:

```
systemctl status bacnet-schub-hub
journalctl -u bacnet-schub-hub -f
sudo systemctl restart bacnet-schub-hub      # after editing hub.conf
```

`systemctl stop` sends SIGTERM, which stops the hub cleanly, like pressing q.

3. Quick start

From a folder where you unpacked the program:

```
# 1. Make a lab certificate set: a CA, the hub's certificate, and 3 device
#   certificates (clients/client-01 .. client-03).
BACnetExampleBSCHUB --generate-certs
```

```
# 2. Start the hub.
BACnetExampleBSCHUB
```

1. Open <http://127.0.0.1:8080/> in a browser. The status page shows the version, whether the BACnet/SC hub is listening, and the connected devices.
2. Connect a device: open <http://127.0.0.1:8080/setup>, the [set-up guide](#). It signs the device's CSR or makes its files, says where each file goes, and shows live why a connection attempt failed. Or give each device one `certs/clients/<label>/` folder by hand. In the [CAS BACnet Explorer](#), import that folder's `bacnetsc.config` (see [Connecting devices](#)).

On Windows the program is `BACnetExampleBSCHUB.exe`. A typical start-up:

```
BACnet B-SCHUB (BACnet/SC Hub) Example - C++ v1.5.0
CAS BACnet Stack version: 6.0.23.0
Common helper (common/) version: 3.0.0
FYI: Listening for BACnet/IP on UDP port 47808 (Network Port 1).
FYI: Device 389022 ("Chipkin Example B-SCHUB") ready. Vendor ID 389. Press 'h' for help, 'm' for a
health/metrics snapshot.
BACnet/SC: listening for WebSocket/TLS connections on wss://0.0.0.0:4443/ (subprotocol
"hub.bsc.bacnet.org", TLS 1.3, mutual auth)
```

At start-up the hub also checks its certificates and logs each one's subject, days until expiry (a warning under 30 days), whether the private key matches, and the subjectAltName entries.

Console keys

Key	Action
h	Help and version.
m	Health and metrics snapshot.
up / down	Change Analog Input 1 by 1.1.
q	Quit.

4. Certificates

BACnet/SC runs over TLS with **mutual authentication**: the hub and every device present a certificate, and each side accepts the other only if that certificate was signed by an issuer (CA) it trusts.

Lab certificates

The hub can make a complete lab certificate set itself:

```
BACnetExampleBSCHUB --generate-certs          # CA + hub + 3 device certificates
BACnetExampleBSCHUB --generate-certs 10      # ... or any number of devices
```

This writes PEM files to --sc-cert-dir (default ./certs) and exits. Easier still: while the hub runs, the [set-up guide](#) at <http://127.0.0.1:8080/setup> signs a device's request or makes its files, and shows every field in them.

The CARI file layout

The files are laid out in the **CARI** format (Certificate Authority Requirements Interchange, ANSI/ASHRAE 135-2024 Annex AA.2, added to 135-2020 by Addendum cs). CARI is the folder layout BACnet uses to take certificate signing requests to a CA and bring the certificates back. Its root folder is cert1/:

- **Folders:** one device-<instance>/ folder per device, and one port-<id>/ folder per BACnet/SC port in it. A hub/ folder in a port folder marks a hub function, and a router/ folder in a device folder marks a device that routes between BACnet/SC networks. Both are empty.
- **Files in a port folder:** csr-<name>.pem (the request), opr-<name>.pem (the operational certificate the CA returns) and, optionally, key-<name>.pem (the private key).
- **The issuer:** cert1/issuer/iss-1.pem (and iss-2.pem).

The hub's own folder is a CARI tree for the hub itself:

File	What it is
cert1/device-<deviceID>/port-2/hub/	Empty: Network Port 2 is a hub function.
cert1/device-<deviceID>/port-2/opr-hub.pem	The hub's certificate (File 1, Operational_Certificate_File). Its subjectAltName lists localhost, 127.0.0.1, this computer's host name and the hub URI's host.

File	What it is
cert1/device-<deviceID>/port-2/key-hub.pem	The hub's private key. Private.
cert1/device-<deviceID>/port-2/csr-hub.pem	CSR for the hub's key (File 2, Certificate_Signing_Request_File). Zip cert1/ and you have a CARL request for your site CA.
cert1/issuer/iss-1.pem	The lab CA (File 3, Issuer_Certificate_Files). Every device needs a copy.
cert1/issuer/iss-2.pem	File 4. Only once a second issuer is added (over BACnet or POST /certs/issuer2); until then File 4 serves iss-1.pem.
ca/ca-cert.pem, ca/ca-key.pem	The lab CA that signs devices (--add-client-certs, --sign-csr, the set-up guide). The key is private ; keep it off the network. Kept outside cert1/, which may only hold CARL files.
key-hub-pending.pem	Only after a GENERATE_CSR_FILE: the new key behind csr-hub.pem, until a certificate for it is activated. Private.
trusted-issuers.pem	Written by the hub: every issuer from both slots, which is what TLS trusts.
issuer-crl.pem	Optional, from your CA: revoked device certificates (see Security).
certificates.txt	Every certificate's label, location, serial number, expiry and SHA-256 fingerprint.
readme.txt	A walkthrough of the folder, including which files are private.
clients/<label>/	One folder per device - below.

Each device folder is that device's CARL response, plus files for the tools that use it:

File in clients/<label>/	What it is
cert1/device-<n>/port-<id>/opr-<label>.pem	The device's certificate (its Operational_Certificate_File).
cert1/device-<n>/port-<id>/key-<label>.pem	Its private key, only when the hub made it. Private. A device that sent its own CSR keeps its key.
cert1/device-<n>/port-<id>/csr-<label>.pem	The request the certificate was signed from.
cert1/issuer/iss-1.pem	The issuer, so the device can validate the hub (its Issuer_Certificate_Files).
<label>-cari-response.zip	cert1/ zipped, for tools that import CARL files.
bacnetsc.config	Import into the CAS BACnet Explorer: the hub URI and the files above (relative paths).
<label>.pfx, iss-1.cer, yabe-bacnetsc.config	For YABE and other Windows tools: certificate + key + issuer (empty password, private), the issuer in DER, and YABE's channel file. Only when the key is in the folder.

The device instance and port id in the folder names default to the client number and 1. Set them with --cert-device-instance and --cert-port-id. Device certificates also carry the label in their subject Common Name (Chipkin Example B-SCHUB client-01), so the hub's log shows which device connected.

Adding devices later. Sign more device certificates with the *existing* CA. Numbering continues, and a running hub trusts them straight away:

```
BACnetExampleBSCHUB --add-client-certs 2 # clients/client-04, client-05
BACnetExampleBSCHUB --add-client-certs 3 --cert-label ahu # clients/ahu-01 .. ahu-03
```

Signing a CARI request. A site's request for several devices, made with BACnet International's BACCARI tool or a vendor's own, is signed in one go. The response is written next to the request:

```
BACnetExampleBSCHUB --sign-csr site-request.zip # -> site-response.zip
```

The response keeps every file of the request. It also gets:

- `opr-<name>.pem` next to each CSR it signed;
- `cert1/issuer/iss-1.pem` (and `iss-2.pem`, if the hub has two issuers);
- `response-notes.txt`;
- `errors.txt`, with one tab-separated line (`device-<n>`, `port-<id>`, reason) for each request it refused.

The command exits with 1 if any request was refused. Before anything is unpacked, the zip is checked against the CARI rules:

- only CARI names, legal characters, and device instances 0 to 4194302;
- no `..` or absolute paths;
- vendor-data up to 1 MB and the notes up to 10 kB;
- the whole file up to 4 MB.

A port folder marked `hub/` gets a certificate with `serverAuth` as well as `clientAuth`.

Signing one device's own CSR. When a device makes its own key and sends a bare certificate signing request (PEM or DER), sign it the same way. The private key never leaves the device:

```
BACnetExampleBSCHUB --sign-csr ahu7.csr --cert-label ahu-7 --cert-device-instance 7007 # clients/ahu-7/
```

The hub first checks that the request's signature verifies and that its key is ECDSA P-256 or stronger, or RSA 2048-bit or stronger.

- **What `clients/ahu-7/` gets:** the device's CARI response (with no key- file), `bacnetsc.config` and `iss-1.cer`. Send that folder, or its `ahu-7-cari-response.zip`, back to the device's owner.
- **What the certificate contains:** the request's subject and public key. Everything else comes from the device profile: 825 days, ECU `clientAuth`, no `subjectAltName`. Any extensions the request asks for are ignored.
- **Defaults:** without `--cert-label` the folder is the next `client-NN`.
- **No restart:** a running hub trusts the new certificate straight away.

To make a key and a CARI request for a device with this program, for example on the device owner's own computer:

```
BACnetExampleBSCHUB --generate-csr --cert-label ahu-7 # clients/ahu-7/cert1/... + ahu-7-cari-request.zip
```

`--generate-csr` needs no CA. Signing that folder in place (`--sign-csr clients/ahu-7`) puts the certificate next to its key, and the folder also gets the `.pfx` and YABE files.

Older folders. A folder made by an earlier release keeps working unchanged. That covers the 1.4 names (operational-certificate.pem, private-key.pem, issuer-certificate.pem, ...) and the older ones (hub.crt, hub.key, ca.crt). The start-up log names the layout it found. --migrate-certs copies such a folder into the CARL layout and leaves the old files as they are, so going back only means deleting cert1/ and ca/.

Starting over. --generate-certs won't replace an existing CA, because every certificate already handed out would stop working. Add --force to delete the whole set, including clients/, and make a new one.

The lab profile is ECDSA P-256 with SHA-256. The CA is valid for 10 years and other certificates for 825 days.

Production certificates

For a production site, use your own CA: have it sign the hub's CSR (File 2), and install the result on disk or [over BACnet](#). [production-certificates.md](#) walks through it: using a site or enterprise CA (root and intermediate), getting the hub's certificate signed, rotating certificates and changing the CA without an outage, revocation, what the certificates should contain, and protecting private keys.

Password-protected private keys

A certificate package may hold an encrypted private key (the file starts -----BEGIN ENCRYPTED PRIVATE KEY-----). The hub asks for its password **once**, when it starts, with the typing hidden, and checks it by decrypting the key; a wrong password is asked for again, up to three times. It keeps the password in memory and uses it everywhere the key is needed - the listener, the connector's every connect attempt, the HTTPS status page - so it is never asked for again during the run.

A service has no console to ask on: set the password in the config file as sc-key-password (config file only, like dcc-password; restrict the file's permissions as described in [Configuration file](#)). When it is set, the hub doesn't ask.

The same password unlocks the [set-up guide](#)'s signing from any computer that reaches it over HTTPS. Without one, signing works only on the hub's own computer.

5. Configuration

BACnetExampleBSCHUB [options]

Options

Option	Default	Meaning
--port <n>	47808	BACnet/IP UDP port.
--bacnet-ip <on/off>	on	off runs BACnet/SC only - see BACnet/SC only .
--deviceID <n>	389022	BACnet device instance.
--device-name <name>	Chipkin Example B-SCHUB	The Device's Object_Name. Must be unique on the BACnet internetwork, so name each hub. 1 to 128 bytes of UTF-8; for a non-ASCII name use device-name in a config file saved as UTF-8.
--ip-network-number <n>, -sc-network-number <n>	not set	Network_Number of Network Port 1 / 2 (1..65534), reported with quality configured. Unset ports report 0, quality unknown.

Option	Default	Meaning
--log-file <path>	none	Also write everything the hub prints to this file (see Logging).
--log-max-size-mb <n>, --log-max-files <n>	10, 5	Rotate the log file at this size, keeping this many old files (<path>.1, .2, ...).
--sc-port <n>	4443	BACnet/SC hub (wss://) port.
--sc-cert-dir <dir>	./certs	Certificate folder.
--sc-hub-uri <wss://host:port/>	off	Also connect out to this BACnet/SC hub.
--sc-failover-uri <wss://host:port/>	off	Failover hub for --sc-hub-uri.
--sc-max-hub-connections <n>	4	Maximum BACnet/SC devices connected at once, 1 to 4 (see Connection limit).
--sc-rate-limit <n>	10	Maximum new BACnet/SC connection attempts per second from any one source address (0 = no limit). Excess attempts are refused before the TLS handshake.
--sc-rate-limit-total <n>	50	Maximum new BACnet/SC connection attempts per second for the whole listener (0 = no limit).
--sc-accept-hub-without-hello	off	Compatibility: let --sc-hub-uri connect to a hub that omits the Hello option. See BACnet/SC compatibility .
--sc-accept-device-without-hello	off	Compatibility: accept a device whose Connect-Request omits the Hello option, such as YABE. See BACnet/SC compatibility .
--http-port <n>	8080	Status page and HTTP endpoints.
--http-bind <addr>	127.0.0.1	Interface for the HTTP listener. See Security before changing it.
--http-tls	off	Serve the HTTP endpoints over HTTPS.
--http-tls-cert <file>, --http-tls-key <file>	the hub's operational certificate and key	Certificate and key for --http-tls.
--sc-keylog-file <file>	off	Debugging only. Append each BACnet/SC connection's TLS session secrets to <file>, so Wireshark can decrypt a capture. Command line only. See Decoding BACnet/SC traffic in Wireshark .
--config <path>	none	Read settings from a config file (below).
--generate-certs [n]	3	Make a lab certificate set (CARL layout) with n device folders, then exit.
--add-client-certs [n]	1	Sign n more device folders with the existing CA, then exit.
--generate-csr	-	Make a private key and a CARL request for one device in clients/<label>/, then exit.

Option	Default	Meaning
<code>--sign-csr <file></code>	-	Sign with the existing CA, then exit: a CARL request zip (writes <code><name>-response.zip</code>), a bare CSR (writes <code>clients/<label>/</code>), or a <code>--generate-csr</code> folder (signed in place).
<code>--migrate-certs</code>	-	Copy an older release's certificate folder into the CARL layout, then exit.
<code>--cert-label <label></code>	client	Device folder name: a prefix with <code>--generate-certs/--add-client-certs</code> , the exact name with <code>--generate-csr/--sign-csr</code> (default: the next <code>client-NN</code>).
<code>--cert-device-instance <n>, --cert-port-id <id></code>	client number, 1	The device's CARL folder names, <code>device-<n>/port-<id></code> .
<code>--inspect <file></code>	-	Say what a certificate, CSR, key, .pfx or CARL zip is, list every field and check it against this hub (the set-up guide's file inspector), then exit; 1 if a check fails.
<code>--cert-hub-uri <wss://host:port/></code>	this computer's IPv4 and <code>--sc-port</code>	Hub URI written into each device's <code>bacnetsc.config</code> (and the hub certificate's <code>subjectAltName</code>).
<code>--force</code>	-	With <code>--generate-certs</code> : replace an existing certificate set.
<code>--install-service, --uninstall-service</code>	-	Windows: set up or remove the BACnetSCHub service (see Running as a service).
<code>--help, --version</code>	-	Usage, or version information.

An unknown option, a stray argument or an option missing its value is an error. The on/off switches (`--http-tls`, `--sc-accept-hub-without-hello`, `--sc-accept-device-without-hello`) mean "on" alone and also take on or off, so the command line can turn off what the config file turned on.

Configuration file

`--config <path>` reads `key = value` lines. A line starting with `#` is a comment; a `#` later in a line is part of the value, so passwords and tokens may contain one. `example.conf` (shipped with the program) lists every key with its default: `device-id`, `device-name`, `ip-network-number`, `sc-network-number`, `log-file`, `log-max-size-mb`, `log-max-files`, `port`, `bacnet-ip`, `sc-port`, `sc-cert-dir`, `sc-hub-uri`, `sc-failover-uri`, `dcc-password`, `sc-key-password`, `http-upload-token`, `http-port`, `http-bind`, `http-tls`, `http-tls-cert`, `http-tls-key`, `sc-max-hub-connections`, `sc-rate-limit`, `sc-rate-limit-total`, `sc-accept-hub-without-hello` and `sc-accept-device-without-hello`. On/off keys take `true/false` (or `on/off`, `yes/no`, `1/0`). A command-line option always wins over the config file. An unknown key or bad value is logged and skipped.

dcc-password, sc-key-password and http-upload-token can only be set in the config file, so they never show up in process listings or shell history. `sc-key-password` is the password of an encrypted private key (see [Password-protected private keys](#)). `dcc-password` protects DeviceCommunicationControl and ReinitializeDevice; `http-upload-token` is the separate secret for the HTTP certificate upload (use a different value - the hub warns if they match). Both are compared in constant time. Restrict the file's permissions (`chmod 600 hub.conf`, or on Windows `icacls hub.conf /inheritance:r /grant:r *S-1-5-18:F *S-1-5-32-544:F "%USERNAME%:F"`, which keeps access for SYSTEM and Administrators; if the installer set up the service, also add "NT SERVICE\BACnetSCHub:R", the account it runs as); the hub warns at start-up if the file looks readable by other users. The Windows installer already restricts

C:\ProgramData\Chipkin\BACnetSCHub to SYSTEM, Administrators and the service's own account, NT SERVICE\BACnetSCHub.

Logging

Everything the hub prints goes to the console. With `--log-file <path>` (config: `log-file`) it also goes to that file, which is appended to across restarts. When the file reaches `--log-max-size-mb` (default 10 MB) it is renamed `<path>.1` (the previous `.1` becomes `.2`, and so on) and a new file is started; `--log-max-files` (default 5) old files are kept. Every line written through the hub's logger starts with a UTC timestamp and a level ([INFO], [WARN], [ERROR]). Connection audit lines start with `SC audit:`.

BACnet/SC only

Some sites want no unencrypted BACnet traffic at all. `--bacnet-ip off` (config: `bacnet-ip = off`) turns BACnet/IP off: the hub opens no UDP port, Network Port 1 doesn't exist (it isn't in the Device's `Object_List`), and the device is reachable only over BACnet/SC - discovery (Who-Is/I-Am) and every other service go through the hub. It doesn't send an I-Am at start-up, since no BACnet/SC device is connected yet; devices find it with Who-Is once they connect.

The hub warns at start-up if BACnet/IP is off and BACnet/SC isn't listening (missing or bad certificates, say), because the device can't then be reached over BACnet at all. BACnet/SC keeps retrying every 5 seconds, and `/health` reports degraded until it is listening; the status page and `/health (bacnet_ip_enabled)` show which mode the hub is in.

BACnet/SC compatibility

ANSI/ASHRAE 135-2024 (clause AB.2.2) requires the BACnet/SC Connect-Request and Connect-Accept messages to carry a *Hello* option. By default the hub follows the standard strictly and refuses a connection that leaves it out.

`--sc-accept-hub-without-hello` (config: `sc-accept-hub-without-hello = true`) relaxes this for the **hub connector** only: when the hub connects out to another hub (`--sc-hub-uri`), it accepts that hub's Connect-Accept without Hello and treats the other hub as having no optional capabilities. Turn it on only for a hub that needs it; the hub logs a warning at start-up while it is on.

`--sc-accept-device-without-hello` (config: `sc-accept-device-without-hello = true`) does the same for **devices** connecting to this hub: it accepts a Connect-Request without Hello, which YABE sends, and records the device as having no optional capabilities. Without it, such a device is refused and the hub logs Connect-Request refused by the hub: "Connect messages require the Hello destination option". Turn it on only while a device like that needs to connect; the hub logs a warning at start-up while it is on.

The two settings are independent; either, both or neither can be on.

Connection limit

This example accepts **at most 4 BACnet/SC devices at a time**. It is for evaluation and testing, not for production. You can lower the limit with `--sc-max-hub-connections` or `sc-max-hub-connections` in the config file, but not raise it: asking for more than 4 prints an error and the hub shuts down.

For a production BACnet/SC hub with more connections, contact Chipkin at support@chipkin.com.

6. Connecting devices

Connecting a device with the set-up guide

The easiest way to connect a device is the hub's **certificate set-up guide**. On the hub's computer, open <http://127.0.0.1:8080/setup> (the status page links to it). It takes you through four steps:

1. **How it works.** What a device needs, and which file goes where. This step shows the URI devices dial, and the hub's and the CA's certificates with every field and every check. Download `iss-1.pem` (or `iss-1.cer` for Windows tools) here: every device needs it.
2. **Get the device's files**, one of two ways:
3. **"My device made a CSR"** (recommended - the device's key never leaves it). Drop in the CSR the device exported, PEM or DER, or paste it, or use a CARL request zip from BACCARI or a vendor tool. The guide says what the file is, shows every field, and checks it. Give the device a folder name, its BACnet device instance and port id, then **Sign**. A CARL request is answered with a CARL response zip.
4. **"Make everything for me"**. For tools that can't make their own key, such as YABE or test tools, the hub makes the key, the CSR and the certificate.

You can then download each file, or the whole folder as a CARL response zip. The device folder is also kept as `clients/<label>/`, so the files can be downloaded again later from **Device folders**. 3. **Install**. Which file goes into which Network Port property of the device: `opr-*.pem` into `Operational_Certificate_File`, `iss-1.pem` into `Issuer_Certificate_Files`, the key, and the hub URI. 4. **Test the connection**. Start the device. The guide shows, live, what the hub sees, and turns each failure into a sentence and a fix: - a TCP connection with nothing after it: the device sent no certificate, only speaks TLS 1.2, or doesn't trust the hub's certificate; - a refused certificate: OpenSSL's reason, such as signed by another CA, expired, not yet valid, no clientAuth, or revoked; - the wrong WebSocket subprotocol; - a refused Connect-Request (duplicate VMAC, missing Hello); - and finally, "Connected!" with the device's VMAC and UUID.

Two tools are always available:

- **File inspector**. Drop in any certificate, CSR, key, `.pfx` (empty password) or CARL zip. It shows every field and checks it against this hub; nothing is stored. The same inspector runs on the command line as `BACnetExampleBSCHUB --inspect <file>`, which exits with 1 if a check fails.
- **Diagnostic report**. A text summary to send to support: versions, the hub URI, the certificates with their checks, connected devices and recent connection attempts. It contains no keys or passwords.

Who may sign. Signing a CSR, making a device's key and downloading a private file need the **password of the hub's private key**: the config file's `sc-key-password`, or the password typed when the hub started (see [Password-protected private keys](#)). With the lab set, whose key has no password, these actions work only on the hub's own computer (from 127.0.0.1). Everything else in the guide is open, like the status page. See [Security](#).

Connecting a device by hand

For each BACnet/SC device:

1. Give it its own `certs/clients/<label>/` folder. Don't share a folder between devices: the hub couldn't tell them apart.
2. **CAS BACnet Explorer**: import the folder's `bacnetsc.config`. It names the hub URI and the folder's certificate, key and issuer files by relative path, so keep the folder together.
3. **YABE (Yet Another BACnet Explorer)**: in *Communication Channel* -> *BACnet/Secure Connect*, **Select** the folder's `yabe-bacnetsc.config`, then **Start**. It points at the folder's `<label>.pfx` (certificate, key and issuer,

with an empty password - keep it private) and `iss-1.cer` by absolute path, so re-select it if you move the folder. YABE uses Windows' own TLS, which can't make the TLS 1.3 connections BACnet/SC requires on Windows 10: use Windows 11 / Server 2022 or later ([#39](#)).

4. **Any other BACnet/SC device:** install `opr-<label>.pem` and `key-<label>.pem` (in `cert1/device-<n>/port-<id>/`) as its operational certificate and key, and `cert1/issuer/iss-1.pem` as its issuer certificate. Set its primary hub URI to `wss://<hub address>:4443/`. A tool that imports CARL files can take `<label>-cari-response.zip` as it is.

The hub URI in `bacnetsc.config` is this computer's LAN address unless you generated the set with `--cert-hub-uri`. The hub logs refused TLS handshakes with the reason, and keeps an audit trail of every connection (lines start with `SC audit:`): the device's address and the subject of the certificate it presented, then its BACnet/SC VMAC and device UUID once the hub accepts it, and the same identity again when it disconnects:

```
2026-09-26 18:10:21 [INFO] SC audit: peer "wss://0.0.0.0:4443/|client=3" connected from
10.0.0.31:64150, certificate "O = Chipkin Automation Systems (lab test), CN = Chipkin Example B-SCHUB
client-01"
2026-09-26 18:10:21 [INFO] SC audit: peer "wss://0.0.0.0:4443/|client=3" (10.0.0.31:64150) is BACnet/SC
device VMAC 0a:0b:0c:0d:0e:0f, UUID 50515253-5455-5657-5859-5a5b5c5d5e5f, certificate "O = Chipkin
Automation Systems (lab test), CN = Chipkin Example B-SCHUB client-01" - connected
```

The status page lists the devices connected right now the same way.

7. Managing certificates over BACnet

A BACnet client can change the hub's certificates with the BACnet/SC certificate procedures (ANSI/ASHRAE 135 clause 19.8.3), for example from the [CAS BACnet Explorer](#)'s certificate page:

1. **Write** - `WriteProperty File_Size = 0`, then `AtomicWriteFile` the new PEM certificate: File 1 for the hub's own certificate, File 3 or 4 for an issuer.
2. **Staged** - reading the File back shows the new certificate and Network Port 2's `Changes_Pending` is `TRUE`, but nothing is saved yet and the hub keeps using its current certificates.
3. **Activate** - `ReinitializeDevice ACTIVATE_CHANGES` (or `WARMSTART`). The hub checks the staged certificates first: each must be a PEM certificate, at least one issuer must remain, and the hub certificate must match its private key and chain to an issuer. If they pass, it saves them and restarts BACnet/SC; devices reconnect under the new certificates. If not, it answers `INVALID_CONFIGURATION_DATA` and nothing changes, so a mistake can't lock the hub out. The staged writes stay, so the client can correct them and activate again.

Supported procedures:

- **Add an issuer** - write the new CA into slot 2 (File 4). Devices signed by either CA are then accepted, which lets a site move to a new CA gradually.
- **Replace the hub certificate** - read the CSR (File 2), have your CA sign it, write the result to File 1, activate.
- **Replace the hub certificate with a new key pair** - write `GENERATE_CSR_FILE` (9) to Network Port 2's Command. The hub makes a new key pair and a new CSR (File 2) with the same subject, saving the new key as `key-hub-pending.pem`; it keeps using its current key and certificate meanwhile. Have the new CSR signed, write the result to File 1 and activate: the new key then replaces `key-hub.pem`. Refused with `INVALID_VALUE_IN_THIS_STATE` while certificate writes are staged - and, until [#41](#) is fixed, on a freshly started hub, which reads `Changes_Pending TRUE` with nothing staged: send `ReinitializeDevice ACTIVATE_CHANGES` once

first. If the hub stops between saving the new certificate and swapping the key, it finishes the swap at the next start.

- **Discard staged writes** - write DISCARD_CHANGES (1) to Network Port 2's Command. The staged certificate writes are dropped, the File objects read the files on disk again, and Changes_Pending goes back to FALSE. A restart also discards staged writes. Until #41 is fixed, don't send it to a freshly started hub: it also reverts the stack's start-up state, after which Changes_Pending no longer follows certificate writes (restart the hub to recover).

Known issue: right after the hub starts, Network Port 2 reads Changes_Pending = TRUE (and Current_Health may report invalid-configuration-data) although nothing has been written. The CAS BACnet Stack leaves the hub's own start-up certificate settings pending (cas-bacnet-stack#2866, #41). The certificates in use are correct. The first ReinitializeDevice ACTIVATE_CHANGES clears it.

If dcc-password is set, ReinitializeDevice requires it.

8. Status page and HTTP endpoints

The hub serves HTTP on --http-port (default 8080), on 127.0.0.1 only by default. With --http-tls it serves HTTPS instead (TLS 1.2 or 1.3), using the hub's own certificate and key (opr-hub.pem, key-hub.pem) unless --http-tls-cert/--http-tls-key name another certificate - for example one your browsers already trust. With the lab certificates, point curl at the lab CA: curl --cacert certs/cert1/issuer/iss-1.pem https://localhost:8080/health (with Windows' own curl, add --ssl-no-revoke: the lab CA publishes no revocation list for Windows to check). The HTTPS certificate is loaded at start-up.

Path	What it returns
GET /	Status page for a browser: versions, device, health, BACnet/SC state, metrics, the connected devices (address, certificate, VMAC and UUID), and links to this project and the CAS BACnet Stack. Refreshes every 5 seconds.
GET /setup	The certificate set-up guide , and the /api/... endpoints it uses (below).
GET /health	<i>Is the hub working?</i> JSON with status ok (HTTP 200) or degraded (HTTP 503 - the BACnet/SC hub isn't listening, usually because of missing or bad certificates). Point uptime monitors and load balancers here.
GET /metrics	<i>How much is it doing?</i> JSON counters: uptime, connected devices, connects, disconnects, rate-limit refusals, messages and bytes in and out, and connections closed because the device stopped reading (sc_tx_queue_overflows).
POST /certs/<slot>	Upload a certificate file (operational, csr, issuer1, issuer2). Needs Authorization: Bearer <http-upload-token>; disabled when no http-upload-token is set. At most 5 attempts a minute from one address and 30 in total; more get HTTP 429 and a log line.

```
$ curl -s http://127.0.0.1:8080/health
{"status":"ok","version":"1.5.0","uptime_seconds":154,"bacnet_ip_enabled":true,"sc_hub_function_listeni
ng":true,"sc_hub_connections_current":1,"staged_certificate_changes":false,"tls_keylog":false}
```

```
$ curl -s http://127.0.0.1:8080/metrics
{"uptime_seconds":154,"uptime":"2m
34s","sc_hub_connections_current":1,"sc_hub_connections_max":4,"sc_total_connects":3,"sc_total_disconne
```

```
cts":2,"sc_rate_limit_rejections":0,"sc_rx_messages":12,"sc_rx_bytes":456,"sc_tx_messages":12,"sc_tx_bytes":456,"sc_tx_queue_overflows":0}
```

Uploading a certificate

An upload is checked exactly like a certificate written [over BACnet](#) before anything is saved: it must be a PEM certificate, and the certificates as they would be afterwards must still let the hub run (the hub certificate matches its private key and chains to an issuer, and at least one issuer remains). If they pass, the file is written atomically and BACnet/SC reloads with it straight away; devices reconnect. If not, the hub answers HTTP 400 with the reason and nothing changes. An upload to `csr` must be a certificate signing request for the hub's own private key. While a certificate change made over BACnet is waiting for `ACTIVATE_CHANGES`, uploads are refused with HTTP 409.

```
curl -X POST --data-binary @new-hub-cert.pem \
  -H "Authorization: Bearer $UPLOAD_TOKEN" http://127.0.0.1:8080/certs/operational
```

Set-up guide API

The [set-up guide](#) is a page in front of these endpoints, which also work with curl. Request bodies are the raw file (PEM, DER or zip), not a form. Answers are JSON except downloads and the report. **Privileged** endpoints need the hub key's password (see [Security](#)), sent as `Authorization: HubKey <base64 of the password>`. When the key has no password, send `Authorization: HubKey -`: the header itself is required, because it is what keeps other web sites from using the hub.

Path	What it does
GET /api/setup/info	The hub URI(s) a device dials, the hub's and the CA's certificates with every field and check, whether the hub can sign, and whether this caller may.
GET /api/setup/issuer/iss-1.pem (iss-1.cer, iss-2.pem)	The issuer certificate(s) every device needs. Public.
POST /api/inspect?name=<file>	What a file is (certificate, CSR, key, .pfx with an empty password, CRL, CARL zip), every field, and the checks against this hub, each with a fix. Nothing is stored.
POST /api/sign? label=&instance=&port=&name=	Privileged. Sign a CSR into <code>clients/<label>/</code> (folders <code>device-<instance>/port-<port></code>), or a CARL request zip into a CARL response (kept in <code>cari-responses/</code>). Answers with what was signed or refused, and download links.
POST /api/generate? label=&instance=&port=	Privileged. Make a device's key, CSR and certificate: a full device folder.
POST /api/check-password	Privileged. 200 if the password is right. Counts as an attempt.
GET /api/clients	The device folders and their files.
GET /api/download/<label>/<file>, GET /api/cari/<file>	A file from a device folder or a CARL response. Private files - <code>key-*.pem</code> , <code>.pfx</code> , a zip holding a key - are privileged .
GET /api/diagnostics?after=<n>	The last 100 BACnet/SC connection events (each with a sequence number; after returns only newer ones) and the devices connected now.

Path	What it does
GET /api/report	A plain-text diagnostic report for support: versions, hub URI, certificates and their checks, connected devices, recent events. No keys or passwords.

Sign a device's CSR from the hub's own computer (lab key, no password):

```
curl --data-binary @ahu7.csr -H "Authorization: HubKey -" "http://127.0.0.1:8080/api/sign?label=ahu-7&instance=7007"
```

```
curl -OJ http://127.0.0.1:8080/api/download/ahu-7/bacnetsc.config
```

With a password-protected hub key:

```
curl --data-binary @site-request.zip -H "Authorization: HubKey $(printf %s "$PW" | base64)" \
"http://127.0.0.1:8080/api/sign?name=site-request.zip"
```

9. Security

- **BACnet/SC** uses TLS 1.3 only, with mutual authentication. A device is accepted if its certificate chains to an issuer in File 3 or 4, and isn't revoked (below).
- **Certificate revocation.** Put your CA's certificate revocation list(s) in `issuer-crl.pem` in `--sc-cert-dir` (PEM; one CRL per issuer, concatenated). The hub then refuses any device whose certificate is on a list. It checks the file every 5 seconds: a new or changed file restarts BACnet/SC, every device reconnects, and revoked ones are refused, so no restart is needed. Revocation checking fails closed: while the file exists, a device whose issuer has no CRL in it, or whose CRL is past its next-update date, is refused too, and a file with no readable CRL stops BACnet/SC (/health reports degraded). Keep the CRLs current. Without the file, revocation isn't checked. The start-up log lists each CRL, how many certificates it revokes, and when it expires.
- **Host names aren't checked** when the hub connects out to another hub. BACnet/SC certificates identify devices, not DNS names; the certificate chain is still verified.
- **The HTTP listener** binds to 127.0.0.1 by default. If you need it from another machine, turn on `--http-tls` before `--http-bind`, so the upload token and status data are encrypted (or use an SSH tunnel or a TLS reverse proxy). The hub logs a warning on every start bound off loopback over plain HTTP. `/`, `/health` and `/metrics` have no authentication, even over HTTPS.
- **Private keys** (`key-hub.pem`, `ca/ca-key.pem`, any `key-<label>.pem` and `.pfx`) must stay private. Keep the CA's key off the hub in production.
- **The set-up guide's signing** (`/setup`, `/api/sign`, `/api/generate` and private downloads) adds devices to your BACnet/SC network, so it is gated by the hub key's password (`sc-key-password`, or the one typed at start-up):
- **Comparison:** the password is compared in constant time.
- **Rate limit:** at most 5 wrong passwords a minute from one address, and 30 in total; then HTTP 429, and every refusal is logged.
- **No clear text:** it is never accepted over plain HTTP from another computer, where it would cross the network unencrypted. Turn on `--http-tls` to sign from elsewhere.
- **No password on the key** (the lab set): signing works only from the hub's own computer, to a page loaded as 127.0.0.1 or localhost.
- **Other web sites can't use it:** every privileged request must carry an `Authorization: HubKey ...` header, which another web site's page can't add without a CORS preflight that the hub never answers. A page loaded under any name other than a loopback one (DNS rebinding) is refused too while the key has no password.
- **Audit log:** every signing, key and private download is logged with the address, the subject and the serial number.
- **Separate token:** `http-upload-token` for POST `/certs/<slot>` is a different secret, and unaffected.

- **TLS key log.** `--sc-keylog-file` writes the BACnet/SC session secrets to a file, so that anyone with the file and a capture can read the traffic. It's for debugging only, and is off unless given on the command line (see [Decoding BACnet/SC traffic in Wireshark](#)). `/health's` `tls_keylog` tells a monitor when it is on.
- **Rate limiting:** new BACnet/SC connection attempts are limited per source address (`--sc-rate-limit`, default 10 a second) and for the whole listener (`--sc-rate-limit-total`, default 50 a second), before the TLS handshake. One flooding host uses up only its own allowance, so devices on other addresses can still reconnect. The hub logs each refusal with the address. A device that stops reading is disconnected after 64 messages are waiting for it.
- **Unencrypted BACnet.** BACnet/IP (UDP 47808) is plain text. Turn it off with `--bacnet-ip off` where only BACnet/SC should be used.

To report a vulnerability, see [SECURITY.md](#).

10. Troubleshooting

Symptom	Cause and fix
cannot start listening ... certificate file(s) missing/unreadable	No certificates in <code>--sc-cert-dir</code> . Run <code>--generate-certs</code> , or point <code>--sc-cert-dir</code> at your certificates. BACnet/IP keeps working. <code>/health</code> reports degraded.
<code>lws_create_context</code> failed ... retrying every 5 s	The BACnet/SC port is in use, or a certificate file doesn't load. The hub keeps retrying; fix the cause and it starts listening without a restart.
SC TLS handshake REJECTED - client certificate failed verification	The device's certificate isn't signed by an issuer the hub trusts, has expired, or is revoked (certificate revoked, unable to get certificate CRL when issuer- <code>cr1.pem</code> has no CRL for its issuer, CRL has expired). Check it with <code>openssl verify -CAfile cert1/issuer/iss-1.pem <the device's opr-*.pem></code> (add <code>-crl_check -CRLfile issuer-cr1.pem</code> with a CRL).
private key ... DOES NOT MATCH at start-up	The hub's key and certificate (<code>key-hub.pem</code> and <code>opr-hub.pem</code> , or <code>private-key.pem</code> and <code>operational-certificate.pem</code> in an older folder) are from different sets.
A device won't connect, and you don't know why	Open <code>/setup</code> , step 4 "Test the connection", and start the device: it shows what the hub saw and what to change. The file inspector (or <code>--inspect <file></code>) checks the device's certificate against the hub.
The set-up guide says "Signing is not available here"	The hub key has no password and you're not on the hub's computer: open the guide there (127.0.0.1), or give the key a password and set <code>sc-key-password</code> . With a password, plain HTTP from another computer is refused too: start the hub with <code>--http-tls</code> .
The set-up guide says "The hub can't sign"	There is no CA to sign with (<code>ca/ca-cert.pem</code> + <code>ca/ca-key.pem</code>), or it isn't one of the hub's issuers. With your own PKI, have your CA sign devices instead; the guide's inspector still checks their files.
Devices that connected before an update to 1.5.0 no longer do	The default BACnet/SC port is 4443; before 1.5.0 it was 47819. Either set <code>sc-port = 47819</code> in the config file (or <code>--sc-port 47819</code>), or point the devices (and the firewall rule) at port 4443. The start-up log names the port in use.

Symptom	Cause and fix
A device can't reach the hub	Check the hub URI in its bacnetsc.config, TCP 4443 in the firewall, and whether 4 devices are already connected (the connection limit). The SC audit: lines show what the hub saw.
refusing a WebSocket upgrade ... (HTTP 400)	The device asked for a WebSocket subprotocol other than hub.bsc.bacnet.org. Check its BACnet/SC settings.
Connect-Request refused by the hub	Another connected device has the same VMAC, or the hub is full.
SC rate limit: refusing a new connection	A device (or something else) is reconnecting faster than --sc-rate-limit allows.
ERROR: TOO MANY BACnet/SC CONNECTIONS REQUESTED and the hub exits	sc-max-hub-connections is above 4. Set it to 4 or less. See Connection limit .
/health is degraded and BACnet/IP is off	BACnet/SC isn't listening, so the device is unreachable over BACnet. Fix the certificates, or run with --bacnet-ip on.
Red Error: lines at start-up	Normal CAS BACnet Stack debug output (e.g. the hub hearing its own broadcast I-Am).
A BACnet/SC tool on Windows 10 (e.g. YABE) fails with A call to SSPI failed / a TLS handshake error, and the hub logs nothing	Windows 10's own TLS can't do TLS 1.3, which BACnet/SC requires. The hub warns about this at start-up on Windows 10. Run the tool on Windows 11 / Server 2022 or later, or use a client with its own TLS 1.3 (CAS BACnet Explorer). See #39 .
The Windows service doesn't start	Check log-file in the service's hub.conf, and Windows Event Viewer > Windows Logs > System for Service Control Manager errors.

Decoding BACnet/SC traffic in Wireshark

BACnet/SC is TLS 1.3, so a capture shows only encrypted records. To see the BACnet/SC messages inside, have the hub write each connection's TLS session secrets to a key log file, and give that file to Wireshark:

```
BACnetExampleBSCHUB --sc-keylog-file sc-keys.log
```

1. Start the capture (for example tcp port 4443; on Windows, capture traffic to the same computer with Npcap's loopback adapter), then start the hub with --sc-keylog-file. Only connections made while the option is on can be decoded, because Wireshark needs each TLS handshake.
2. In Wireshark: **Edit > Preferences > Protocols > TLS > (Pre)-Master-Secret log filename**, and select sc-keys.log. With tshark: tshark -r capture.pcapng -o tls.keylog_file:sc-keys.log.
3. The records then decode as WebSocket, and the BACnet/SC frames as **BSCVLC** (Connect-Request, Connect-Accept, Encapsulated-NPDU, ...). Filter with bscv1c. If you run BACnet/SC on a port Wireshark doesn't recognise, use **Decode As > TCP port > TLS**.

The file covers both BACnet/SC roles: devices connecting to the hub, and the hub's own connection to another hub (--sc-hub-uri). It doesn't cover the HTTPS status page. The hub appends to the file and flushes every line straight away, so you can decode a capture while the hub is still running.

Anyone with this file and a capture can read that traffic. Use the option for debugging only, then delete the file. It never holds a private key. It can be set on the command line only (there is no config-file key), so a service can't be

left logging by a forgotten line in its config. While it is on, the hub logs a warning at start-up and whenever BACnet/SC starts listening, /health reports "tls_keylog": true, and the status page shows a banner.

11. The BACnet device

Device 389022	"Chipkin Example B-SCHUB"	Vendor 389 (Chipkin Automation Systems)
└─ Analog Input 1	"Bronze"	example sensor value, degrees C
└─ Network Port 1	"BACnet IP"	BACnet/IP, UDP 47808 (absent with --bacnet-ip off)
└─ Network Port 2	"BACnet SC"	BACnet/SC hub function (and optional connector)
└─ File 1	"Operational Certificate"	the hub's certificate (writable)
└─ File 2	"Certificate Signing Request"	CSR for the hub's key (read-only)
└─ File 3	"Issuer Certificate Slot 1"	trusted CA certificate (writable)
└─ File 4	"Issuer Certificate Slot 2"	second trusted CA certificate (writable)

Every object has a **Description** saying what it is for. The Device's Description links back to the project.

The device instance defaults to 389022 (--deviceID), and the Device's name to "Chipkin Example B-SCHUB" (--device-name). Both must be unique on the BACnet internetwork. Analog Input 1 is example data: the up/down arrow keys change it while the hub runs.

Each Network Port reports Network_Number 0 with Network_Number_Quality **unknown** unless --ip-network-number / --sc-network-number sets it; this device isn't a router. The private key is never served by any object. The [PICS](#) lists every property.

12. Version, licensing and support

- **Version:** this manual describes version 1.5.0. What changed in each release is in [CHANGELOG.md](#); what a version number promises is in [SUPPORT.md](#).
- **Licensing:** the hub's own source code is public domain (CC0-1.0). It is built on the **CAS BACnet Stack**, a commercial Chipkin product, and on OpenSSL (Apache-2.0), libwebsockets (MIT), libuv (MIT), zlib (Zlib) and, on Windows, pthreads4w (Apache-2.0) - see [THIRD-PARTY-NOTICES.md](#). Each release's sbom-*.cdx.json lists the exact versions.
- **Support:** questions and bug reports go to [GitHub issues](#). For commercial support, a production hub or a CAS BACnet Stack licence, contact support@chipkin.com. Security problems: [SECURITY.md](#).
- **Related documents:** the [fact sheet](#), the [PICS](#), the [production certificate guide](#) and the [BTL test plan](#).